

Macroscopic Design of Swarms with Collective State Machines

Gianluca Aguzzi¹[0000–0002–1553–4561], Giorgio Audrito²[0000–0002–2319–0375],
Roberto Girau¹[0000–0002–1298–7960], Gianluca Torta²[0000–0002–4276–7213], and
Roberto Casadei¹[0000–0001–9149–949X]✉

¹ Università di Bologna, Italy
{gianluca.aguzzi, roberto.girau, ✉ roby.casadei}@unibo.it
² Università di Torino, Italy
{giorgio.audrito, gianluca.torta}@unito.it

Abstract. Promoting collective intelligence in artificial collectives, such as robot swarms, is complex and generally addressed with different methods ranging from machine learning to dynamic control. In particular, macro-programming approaches like aggregate computing tackle self-organising spatiotemporal formation and distributed information processing using ad-hoc computational models and domain-specific languages (DSLs) following a macroscopic perspective. However, these approaches come with limited support for (i) effectively specifying the collective phases of a mission, and (ii) ensuring that the collective quickly and consistently reaches an agreement on the collective task to be carried out. To address this, our work proposes the novel notion of *collective finite state machines (cFSM)*, where states and links denote collective tasks and agreements on collective strategy evolution, respectively. Specifically, we provide a general formal characterisation of the approach on event structures, and develop a linguistic implementation in terms of a new DSL layer over the ScaFi aggregate programming language. We also show that our design provides formal guarantees on collective agreement, and its implementation, tested on a swarm robotics case study, is correct, withstands perturbations, and works with bounded memory.

Keywords: macro-programming · collective intelligence · collective computing · distributed systems · coordination models · swarms

1 Introduction

In this work, we are concerned with the engineering of *artificial collective intelligence (ACI)* [22], i.e., the ability of (large) groups of computational agents to solve distributed problems in some environment. Specifically, we address this in terms of *collective computation models and languages* [11,20] able to streamline the software engineering of multi-agent and collective systems [21] (e.g., sensor networks, robot swarms, and cyber-physical ecosystems). We target *distributed settings*, where each agent can only sense and act on a local portion of the environment, perform a local computation, and exchange data with neighbours only.

Specifically, we follow a *macro-programming* approach [4,3], where the “(macro-)program” defining the control logic of the agents focuses on the behaviour of the “entire system” rather than on low-level details of individual agents behaviour.

A common meta-model for the behaviour of agents, also in distributed and swarm settings, is given by finite state machines (FSMs) and corresponding variants [12,13,8]. In such meta-models, the idea is to break the behaviour of some type of agents into different sub-behaviours, each one modelled by a different *state*, and to model behaviour change through *state transitions*. Variants tailored to multi-agent systems include: *probabilistic FSMs* [12], where a macro-level FSM stochastically describes the changes of the average number of agents in each state; and *statecharts* [10], extending FSMs with hierarchical data flows, possibly combining global-level and role-specific statecharts [17].

In this work, we develop a new FSM variant, which we call *collective finite state machine (cFSM)*, aimed at addressing the problem of ensuring that a multi-agent system behaves “as a single entity”. The basic idea is twofold. First, we use states to denote *collective behaviours* rather than individual ones. Second, we define a general strategy for structuring how *consensus* is reached about the current target global state (i.e., the collective computation that needs to be carried out at any time). The contributions are as follows: (i) we formalise the idea of collective finite state machines (cFSMs) for multi-agent systems on the general collective computation model of *augmented event structures* (cf. [2]); (ii) we provide an operational model in the *aggregate computing* macro-programming paradigm [20] and a concrete language implementation on top of the ScaFi language [5]; and (iii) we evaluate the proposal and the implementation on a swarm robotics case study, demonstrating correctness, resilience, and practical history independence (i.e., scalability with respect to the event history size).

The paper is organised as follows. Section 2 covers related work on FSM-based and collective computation models. Section 3 presents the main contribution: the cFSM model and its properties. Section 4 provides an evaluation of cFSMs on a swarm robotics case study. Finally, Section 5 presents concluding remarks.

2 Background and Related Work

2.1 Finite state machines for multi-agent systems and swarms

Finite state machines (FSMs) are a popular model for different kinds of computational processes. Formally, a FSM can be defined (variants exist [17]) as a 4-tuple $M = (S, L, \delta, s_0)$, where: S is a finite set of states; L is a finite set of labels;³ $\delta : S \times L \rightarrow S$ is a deterministic transition function mapping state-label pairs to states; s_0 is the initial state. FSMs can be used as models for the behaviour of multi-agent systems, including robot swarms [9]. In these cases, states usually denote *actions* to be performed (e.g., $s_0 \rightarrow$ “explore an area”, or $s_1 \rightarrow$

³ Labels are often called *events* instead. We opted for this terminology to avoid confusion with the unrelated notion of events in aggregate computing (see Section 2.2).

“come back to headquarters”), and transitions change actions e.g., based on sensory input or information from neighbours. Over the years, multiple variants or extensions of the basic FSM model have been proposed. Research is vast: next, we report representative works where FSMs are used to address macroscopic behaviour.

A general extension is that of *probabilistic FSMs* [19], where initial states and transitions have associated probabilities. In [12], the authors provide a *macroscopic probabilistic model* for collective foraging based on probabilistic FSMs and difference equations. The goal, however, is to optimise a parametrised controller, rather than providing a flexible programming solution as cFSMs.

Statecharts that extend FSM models with *hierarchical data flow* [10] were also proposed for modelling aspects like individual agent plans and interaction protocols [17]. These often link an abstract *global statechart* that describes the entire protocol to role-specific statecharts. Also here, however, the states refer to individuals and not to the whole collective as in cFSMs.

In [15], the *DaNCES framework* aims to link individual to collective behaviour in terms of FSMs where (i) states are further composed of *internal-state control (ISC) units* that can alter the agent’s state properties, and (ii) each agent owns a *parametrised* transition matrix. However, it follows a *data-based approach*, rather than a behaviour-based, programmatic one as in cFSMs.

A different line of works consider *mechanisms* for achieving a sort of “global FSM” where common states are attained through, e.g., barrier synchronisation [18] or coupled oscillators [14]. These may represent *patterns* for cFSMs, which do not fix a given mechanism but aim to provide a common programmatic framework based on the collective computational model described in Section 2.2.

Finally, other meta-models beside FSMs have been proposed for swarm design, such as *BPMN* [6] and *choreographies* [7], but these generally use more complex constructs and target more heterogeneous, multi-party coordination.

2.2 Aggregate Computing and Event structures

aggregate computing (AC) [20] is a macro-programming approach [3] for defining the self-organising behaviour of large groups of situated agents (e.g., swarms).

System model and computation model. An *aggregate system* consists of a set of *nodes* (or *devices*) equipped with *sensors* and *actuators*. Nodes can communicate directly only with *neighbours* (for some *neighbouring relationship*, e.g., proximity-based). Each device is periodically activated to execute a *sense–compute–interact* round involving three steps: (i) **Sense**: the node samples its sensors and gathers asynchronously received messages from its neighbours to define a local context; (ii) **Compute**: the aggregate program (unique for all nodes) is evaluated in the local context to produce exports and actuations; and (iii) **Interact**: the export is broadcast to neighbours and local actuations are performed. Nodes alternate sleep with sense–compute–interact rounds, participating in the global distributed process via application-specific scheduling.

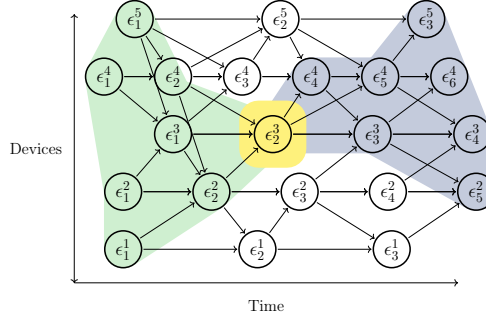


Fig. 1: Example event structure modelling a collective dynamics. Nodes labelled by ϵ_k^δ denote the k -th round of device δ . Given a reference event (yellow), its causal past (green) and future (blue) are determined by the messaging relation.

Aggregate macro-programming model. AC has a *functional* macro-programming model where every device executes the same program (a.k.a. *macro/aggregate program*). The model is based on the *field abstraction*: a field maps devices to values, and hence can be used to denote “collective values” that change in time. It is possible to reason on programs with “global fields” spanning the entire system, but execution will be decentralised, operatively based on “neighbouring fields” only. So, functions can take and return fields, and may include calls to primitives that enable exchange of data with neighbours: for instance, a function `gradient(s)` can take a Boolean field of sources s and return the field of distance estimates to them (e.g., computed by running the adaptive Bellman-Ford algorithm, where local distance estimates are shared with neighbours). Notice that transient failures (e.g., message loss) and changes in the source set and network topology will gradually be absorbed, based on the repeated execution of the program according to the aforementioned execution model, pursuing the stabilisation of the target fields. This programming model is formalised by so-called *field calculi* [20]. For the implementation of our proposal, we leverage the Scala-internal domain-specific language (DSL) SCAF1 [5].

Abstract collective computation model: augmented event structures. We now introduce the notion of *augmented event structure* (see also [2]), which captures the execution semantics of *aggregate systems* in more general terms. This will serve as a basis for formalising our cFSM contribution (Section 3).

Definition 1 (Augmented Event Structure). An augmented event structure is a 4-tuple $\mathbf{E} = \langle E, \rightsquigarrow, d, s \rangle$, where E is a countable set of events, $\rightsquigarrow \subseteq E \times E$ is a messaging relation, $d : E \rightarrow \Delta$ is a mapping from events to the devices where they happened, $s : E \rightarrow S$ is a mapping from events to (some representation of) sensor status, such that:

- for any device $\delta \in \Delta$, the set of events $E_\delta = \{\epsilon \in E \mid d(\epsilon) = \delta\}$ forms a sequence of chains, i.e., there are no distinct $\epsilon, \epsilon_1, \epsilon_2 \in E_\delta$ such that either $\epsilon \rightsquigarrow \epsilon_i$ for $i = 1, 2$ or $\epsilon_i \rightsquigarrow \epsilon$ for $i = 1, 2$;

- the transitive closure of $\rightsquigarrow$ forms an irreflexive partial order $< \subseteq E \times E$, called causality relation;
- the set $X_\epsilon = \{\epsilon' \in E \mid \epsilon' < \epsilon\} \cup \{\epsilon' \in E \mid \epsilon \rightsquigarrow \epsilon'\}$ is finite for all ϵ (i.e., $\rightsquigarrow$ and $<$ are locally finite).

We denote with $N(\epsilon)$ the *neighbours* of ϵ , i.e., the set of events $\{\epsilon' \in E \mid \epsilon' \rightsquigarrow \epsilon\}$. Figure 1 shows an example of an augmented event structure. Referring to the informal computation model introduced above: the events ϵ correspond to the sense–compute–interact rounds; the $\rightsquigarrow$ relation captures messaging with neighbours; fields can also be intended dynamically as maps from events to values.

3 Collective State Machines

We introduce the notion of *collective finite state machine (cFSM)* as a FSM extension where (i) states are *collective*, i.e., denote “collective behaviours” (e.g., aggregate computing functions—cf. Section 2.2) and (ii) transitions are also *collective*, namely meant to be followed by *all*⁴ the agents. Specifically, cFSMs associate a collective program with each state. They also attach priorities to transitions to resolve conflicts, enabling the network to collectively choose which transition to follow. This behaviour is formalised in the following definition.

Definition 2 (cFSM). A cFSM is a tuple $\mathcal{M} = (Pr, S, (pr^*, s^*), P)$, where:

- Pr is a totally ordered set of priorities;
- S is a finite and totally ordered set of states;
- $(pr^*, s^*) \in Pr \times S$ is the initial prioritised state;
- $P : S \rightarrow \mathbf{P}$ maps each state $s \in S$ to a collective program $P(s) \in \mathbf{P}$, whose execution according to an augmented event structure $\mathbf{E}$ produces a prioritised state $(pr', s') \in Pr \times S$ on each event ϵ in $\mathbf{E}$.

Remark 1. Unlike classical FSMs, this definition includes the specific collective program being executed rather than a high-level representation of labels and transitions. This is because we allow for more complex behaviour than direct reactions to inputs. Indeed, a collective program can gather information in various ways to reach a collective decision, computing its priority dynamically. For a subset of programs, we can define an FSM-like transition relation $\tau : S \times L \rightarrow Pr \times S$. This models programs $P(s)$ that, at each event ϵ , compute (zero or more) active labels $l_1, \dots, l_n \in L$ and return the prioritised state $\max\{(pr^*, s), \tau(s, l_1), \dots, \tau(s, l_n)\}$.

Example 1. Consider a scenario of a search and rescue mission by a robot swarm in a military context. We may use a cFSM $\mathcal{M} = (Pr, S, (pr^*, s^*), P)$ where $Pr = \mathbb{R}^\infty$ (real numbers with infinity), $S = \{\text{Wait, Wander, Solve, Defend}\}$, $(pr^*, s^*) =$

⁴ This does not mean that the model only supports “homogeneous behaviour”: within a collective state, it is possible that different (groups of) devices perform different tasks; however, there is *joint agreement on such heterogeneity*.

$(-\infty, \text{Wait})$ with a (default) total order $\text{Wait} < \text{Wander} < \text{Solve} < \text{Defend}$. Many programs $P(s), s \in S$ could be used in this context, choosing new states and priorities according to many different strategies. If the same priority is always returned by programs $P(s)$, the new state is always chosen according to the total ordering on states. Priorities may instead be chosen depending on the rank of the device proposing the transition, to encode a leader-based transition strategy, or they may depend on the current time of the transition, to favour the first transition proposal after later proposals. Generally, the priority may depend on a locally-computed strength of the transition suggestion, e.g., a higher priority assigned to the transition to *Wander* to look for higher-severity targets rather than low-severity nearby targets.

In order to ensure eventual convergence of the network state, it is necessary to compare whole *histories* rather than single prioritised states.

Definition 3 (History). *Given a cFSM $\mathcal{M} = (Pr, S, (pr^*, s^*), P)$, a history h generated by $\mathcal{M}$ is a sequence of prioritised states: $h = [(pr_1, s_1), \dots, (pr_N, s_N)]$ in $Pr \times S$, such that:*

- $(pr_1, s_1) = (pr^*, s^*)$;
- $s_{i+1} \neq s_i$ for all $1 \leq i \leq N - 2$.

In other words, the history cannot contain looping transitions $s_i = s_{i+1}$ except possibly for the last transition from s_{N-1} to s_N .

Example 2. Referring to Example 1, a robot R may transition from *Wait* to *Wander* with priority 10, then after some time (i.e., self-loops in *Wander*) to *Solve* with priority 100, and then back to *Wait* with priority 10. The history associated with this behaviour would be $h = [(-\infty, \text{Wait}), (10, \text{Wander}), (100, \text{Solve}), (10, \text{Wait})]$. Note that self-loops are not kept in the history (except possibly one self-loop at the end, not shown in h).

A history records the sequence of unique prioritised states visited by the network. It filters out self-loops to keep the history length bounded, except at the very end of the sequence. A final self-loop is retained, when present, to encode the priority for remaining in the current state. This special treatment of self-loops must be taken into account when defining the extension of a history with new transitions.

Definition 4 (History Extension). *For history $h = [(pr_1, s_1), \dots, (pr_N, s_N)]$ and prioritised state (pr_{new}, s_{new}) , the extension of h is:*

1. $h' = [(pr_1, s_1), \dots, (pr_N, s_N), (pr_{new}, s_{new})]$, if $N = 1$ or $s_{N-1} \neq s_N$;
2. $h' = [(pr_1, s_1), \dots, (pr_{N-1}, s_{N-1}), (pr_{new}, s_{new})]$, if $N \geq 2$ and $s_{N-1} = s_N$.

Note that, depending on the new prioritised state (pr_{new}, s_{new}) , a history h ending with a self-loop (case 2) can be extended to one that does not (case 1), and vice versa. However, at most one self-loop is ever stored in a history, as required by Definition 3. These definitions allow us to establish a total order on histories: comparing histories h, h' results in either $h < h'$, $h = h'$, or $h > h'$.

Definition 5 (Order of Histories). Let H be the set of possible histories of a cFSM $\mathcal{M}$. We define a total order $<_H$ on H as the lexicographic order where elements (pr, s) are compared first by priority and then by state ordering.

Specifically, two histories $h = [(pr_1, s_1), \dots, (pr_N, s_N)]$ and $h' = [(pr'_1, s'_1), \dots, (pr'_M, s'_M)]$ (with $(pr_1, s_1) = (pr'_1, s'_1) = (pr^*, s^*)$) are compared in *lexicographic order*⁵, by comparing elements (pr_i, s_i) and (pr'_i, s'_i) first by priority and then by state, which are both totally ordered by Definition 2.

To ensure eventual consistency of the history across the network, the ordering always favours larger histories. This allows us to define the distributed computation of a cFSM $\mathcal{M}$ on an augmented event structure $\mathbf{E}$, as follows.

Definition 6 (cFSM Execution). Let $\mathcal{M} = (Pr, S, (pr^*, s^*), P)$ be a cFSM and $\mathbf{E}$ be an augmented event structure. For each event $\epsilon \in E$, we define:

- the input history: $h_i(\epsilon) = \begin{cases} [(pr^*, s^*)] & \text{if } N(\epsilon) = \emptyset, \\ \max_{\epsilon' \in N(\epsilon)} h_o(\epsilon') & \text{otherwise;} \end{cases}$
- the active state $s(\epsilon) = s_N$, where $h_i(\epsilon) = [(pr_1, s_1), \dots, (pr_N, s_N)]$;
- the output history $h_o(\epsilon)$ obtained by extending $h_i(\epsilon)$ with the prioritised state (pr_o, s_o) produced by executing program $P(s(\epsilon))$ at event ϵ , as in Definition 4.

Example 3. Consider Example 2, where robot R generated history $h = [(-\infty, \text{Wait}), (10, \text{Wander}), (100, \text{Solve}), (10, \text{Wait})]$. Now, suppose the behaviour of another robot R' generated a different history $h' = [(-\infty, \text{Wait}), (10, \text{Wander}), (\infty, \text{Defend})]$. According to Definition 5, we have that $h' > h$, i.e., also robot R , whose behaviour generated h , should adopt it, by transitioning to *Defend*.

3.1 Compacting Histories

Although the basic cFSM ensures eventual convergence on a unique network-level history, it scales poorly. Because histories grow linearly over time, executing the state machine becomes increasingly expensive. Furthermore, keeping unbounded histories is unnecessary: old decisions do not affect convergence once all devices agree on them. This insight motivates a notion of *compressed histories*, whose growth is limited by pruning older prioritised states according to criteria discussed below.

Definition 7 (Compressed History). A compressed history is a 3-tuple:

$$ch = (N_F, [(pr_1, s_1), \dots, (pr_{N_K}, s_{N_K})], [ts_1, \dots, ts_{N_K}])$$

where $N_F \geq 0$ is the number of forgotten prioritised states, $N_K \geq 1$ is the number of kept prioritised states, and ts_i is the timestamp when state s_i was entered through a transition from the previous state.

⁵ Lexicographic order compares sequences by their first differing element, with prefixes ranking lower.

The timestamps ts_i represent times on an approximate shared clock⁶. The extension of compressed histories with a prioritised state (with timestamp) is defined analogously to Definition 4. To define the execution of a cFSM using compressed histories, we also need to uniquely define the maximum of a set of compressed histories. To do so, we first align compressed histories by prefixing forgotten elements. For a compressed history $ch = (N_F, [(pr_1, s_1), \dots, (pr_{N_K}, s_{N_K})], [ts_1, \dots, ts_{N_K}])$, we define its *aligned history sequence* $seq(ch)$ as:

$$seq(ch) = \underbrace{(\dots, (? , ? , ?))}_{N_F \text{ times}}, (pr_1, s_1, ts_1), \dots, (pr_{N_K}, s_{N_K}, ts_{N_K})$$

where $(?, ?, ?)$ represents a forgotten element. The lexicographic comparison extended to forgotten elements is not transitive, hence it is not a total order. Still, it allows for a well-defined notion of maximum.

Definition 8 (Maximum of Compressed Histories). *Let $C = \{ch^1, \dots, ch^n\}$ be a set of compressed histories, and $A^1, \dots, A^n$ be their corresponding aligned history sequences (padded with trailing $(?, ?, ?)$ elements to have the same length M).*

- Two sequences A^i, A^j are compatible if $(pr_k^i, s_k^i) = (pr_k^j, s_k^j)$ at all indices k where both are non-forgotten.
- If there are incompatible sequences, we discard any sequence that has a strictly smaller prioritised state (pr_k, s_k) at the first index k of incompatibility, letting C' be the set of remaining compatible histories.
- The maximum is then defined as the sequence $(pr_1^\top, s_1^\top, t_1^\top), \dots, (pr_M^\top, s_M^\top, t_M^\top)$, where $(pr_k^\top, s_k^\top, t_k^\top)$ is $(?, ?, ?)$ if all remaining $ch^i \in C'$ are such that $(pr_k^i, s_k^i, t_k^i) = (?, ?, ?)$; otherwise $(pr_k^\top, s_k^\top, t_k^\top) = (pr_k^i, s_k^i, t_k^i)$, where $ch^i \in C'$ is such that $(pr_k^i, s_k^i, t_k^i) \neq (?, ?, ?)$ and t_k^i is minimum.

With this setup, cFSM execution on compressed histories computes the input history as the maximum of predecessors' outputs. Its final state determines the program, which appends a new prioritised state (with the current timestamp) to the history while pruning older entries.

Forgetting old transitions is guided by heuristic criteria designed to prevent inconsistencies on forgotten states. The simplest heuristic, used in our experimental evaluation, removes transitions older than the current time by a given threshold ΔT . If any transition is forgotten, we always keep at least two non-forgotten transitions to ensure final self-loops can still be detected. If ΔT is an upper bound on the network reachability time, the forgotten history is guaranteed to be shared across the entire network. In this case, execution with compressed histories produces the same outcomes as with full histories.

⁶ This can be any approximate shared clock [2]. Since it is only used for compression, low accuracy suffices.

4 Experimental Evaluation

The experimental evaluation aims to assess the cFSM approach on three aspects: (i) *correctness*, i.e., whether all agents eventually converge to a consistent collective state; (ii) *resilience*, i.e., the ability to maintain or recover agreement under asynchronous execution and *conflicting events* (i.e., different proposals of target states); and (iii) *practical history independence*, i.e., the ability to operate continuously with a bounded memory footprint (cf. compressed histories). The complete source code, data, and analysis scripts are available at <https://github.com/cric96/experiments-2025-collective-state-machines>.

4.1 Scenario Description

Consider the scenario of a search and rescue mission (e.g., in military context) involving a swarm of N autonomous drones. Initially, all drones stay at the base (e.g., for recharging or maintenance) until an alarm signal is received.

Upon alarm, the swarm is deployed to explore a designated $500 \times 500 \text{ m}^2$ square operational area. Within this area, a target (e.g., a lost soldier) must be located and rescued. The communication capabilities of each drone are limited to a variable radius of R m, while locomotion is constrained to a maximum speed of 10 m/s. Finally, drones are equipped with specialised sensors (e.g., cameras) enabling target detection within a range of 100 m. The same sensing range applies to the detection of attacks on the base station. For collective coordination, each drone maintains a *compressed history* (cf. Section 3.1) over a variable sliding window of H s. To introduce stochastic behaviour and avoid artificial synchronisation among agents, each drone works with a variable round execution frequency (sampled from a Weibull distribution with shape k and scale 1).

Two types of events drive the mission dynamics. First, an *alarm event* triggers the primary mission of finding a target. Second, a *base attack* may occur as an emergency interrupt. The current collective task (i.e., rescuing targets *or* defending the base) is considered complete when all 40 drones converge within 100 m of the relevant location (target or base) and hold their position for 60 s (simulating rescue or defence operations).

Three alarm events are scheduled starting at $t = 1100$ s, $t = 2300$ s, and $t = 3500$ s, simulating repeated search missions. A 100 s slack is introduced between each alarm and the actual target spawn, to ensure that drones have sufficient time to transition into the *Wander* state before the target becomes detectable. Also, a single base attack is triggered at $t = 2400$ s, deliberately coinciding with the active search phase of the second mission. This setup may create a conflict scenario with competing stimuli—finding the target vs. defending the base—enabling an assessment of how the cFSM resolves conflicting state transitions and seeks consensus under concurrent events. For this purpose, base defence is assigned a higher priority than target search.

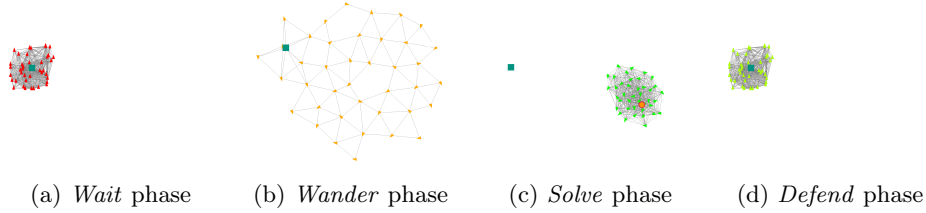


Fig. 2: Visualisation of the swarm behaviour during the four main collective states. The base station is represented by the green square (■) at the top-left corner, while the target is the red circle (●). Drones are depicted as triangles (▲), with their colour indicating the current state: red for *Wait* (▲), orange for *Wander* (▲), light green for *Solve* (▲), and green for *Defend* (▲).

4.2 Simulation Setup & Implementation Details

Simulation Tools. The experimental environment is based on Alchemist [16], a stochastic discrete-event simulator tailored for pervasive computing and multi-agent systems. The collective behaviour and the cFSM logic was implemented in ScaFi [5]. Drone movement was handled by MacroSwarm [1], a library for swarm robotics on top of ScaFi.

Implementation. The implementation realises the scenario described in Section 4.1, with the swarm transitioning among four states: *Wait* (at the base station), *Wander* (search for the target), *Solve* (rescue the target), and *Defend* (return to protect the base) (see Figures 2 and 3). To manage transitions, we employed a priority mechanism: standard transitions use the negated generation timestamp ($-t$) as priority to favor earlier events, which establishes preference when comparing histories via the lexicographic order $<_H$ (Definition 5).

On the other hand, the critical transition to *Defend* is assigned infinite priority ∞ to preempt other tasks, guaranteeing that any device adopting this transition propagates it as the unique network-level maximum under the distributed execution of Definition 6. In the *Solve* state, drones identifying the target broadcast their location using gradients (see Section 2.2). Snapshots of the resulting behaviour are shown in Figure 2.

Experimental Settings. Each configuration was evaluated over 128 independent runs, each lasting 4000 seconds. In each run, the initial positions of the drones and the location of the target are randomly generated, introducing spatial variability to check the results are robust to different topologies. We varied the following parameters: (i) swarm scale, testing both $N = 40$ and $N = 160$ drones to evaluate scalability; (ii) communication radius $R \in \{75, 100\}$ m; (iii) history time window $H \in \{1200, \infty\}$ s; and (iv) the Weibull shape parameter $k \in \{4, 7, 10\}$ to modulate the variability of the round execution rate. Varying the computation rate and its variability also simulates different relative mobilities, as lower execution frequencies lead to larger spatial displacements between rounds.

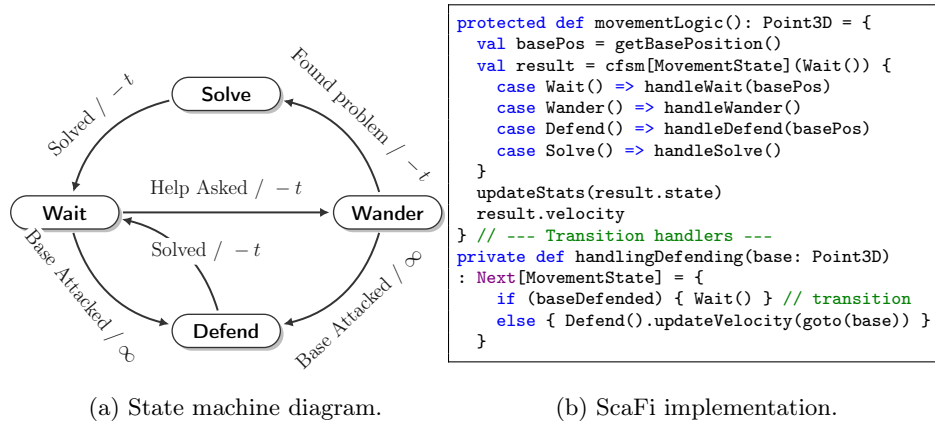


Fig. 3: State machine for the synthetic case study: visual representation (left) and corresponding code (right). Notice that the code is valid Scala and ScaFi code (to be executed repeatedly by all the devices as described in Section 2.2). The main control program is given by function `movementLogic`: it uses the reusable `cfsm` function, whose body maps states to transitions (FSM-like structure) and internally implements the cFSM semantics discussed in Section 3.

Metrics. To evaluate the cFSM approach, we track three time-varying quantities throughout each simulation, namely *state occupancy*, *disagreement rate*, and *history size*. *State occupancy* represents, at each time step, the fraction of agents residing in a given collective state (i.e., *Wander*, *Defend*, *Wait*, *Solve*), averaged across all runs. This measures the active state $s(\epsilon)$ for each device’s event ϵ . The system should eventually reach a state with maximum occupancy (1.0 of agents, indicating consensus). *Disagreement rate* is a binary signal that, at each time step, equals 1 if at least one agent is in a different state than the majority in any of the runs, and 0 otherwise. This tracks transient inconsistencies during state transitions, measuring the convergence rate of the distributed execution toward a unique network-level history. The lower this value is over the runs, the better. *History size* measures, at each time step, the average number of kept prioritised states (N_K) in the compressed histories across all agents (validating the practical history independence of the approach, see Section 3.1).

4.3 Results and Discussion

We analyse the swarm behaviour for the simulated conditions, focusing on how the collective state evolves over time and how it withstands environmental change. The plots in Figures 4 and 5 show the system’s dynamics, illustrating the state occupancy, disagreement rate, and history size across different configurations. We discuss these results w.r.t. the aforementioned evaluation goals.

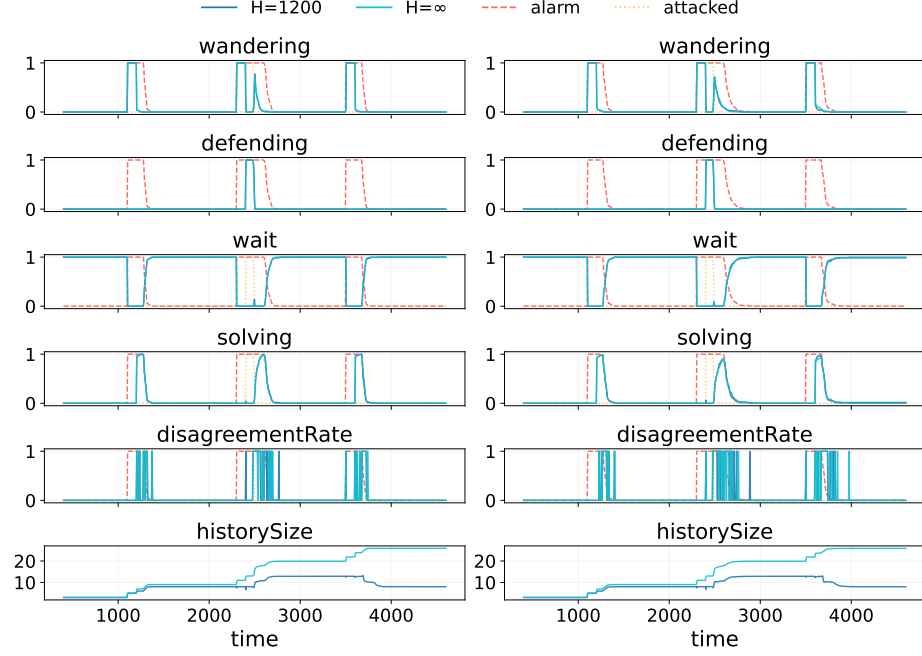


Fig. 4: The plots show the evolution of state distribution, disagreement, and average history size over time, for different history window sizes (with network connectivity of 100m [left] and 75m [right]). Each line represents the average over N_{runs} independent runs. The alarm (red line) indicates that there exists a help request, and the attacked event (yellow line) indicates the base is attacked.

Correctness. The primary goal is to ensure that the swarm collectively transitions to the appropriate state in response to environmental events, maintaining consensus. Figure 5 shows the time evolution of the fraction of agents in each state. Initially, the swarm is in the *Wait* state (third panel). Upon the activation of the *alarm* signal (indicated by the red dashed line), the *Wait* state occupancy drops to zero, and the *Wander* state (top panel) rapidly rises to 1, demonstrating a correct and cohesive transition to the search phase. When the base is attacked (indicated by the yellow dotted line), the system prioritises the *Defend* state (second panel). As observed, the *Defend* state occupancy spikes to 1 during the attack interval, causing the *Wander* and *Solve* states to correctly drop to 0. This confirms that the mutual exclusion between conflicting tasks is preserved and that the swarm effectively handles high-priority interrupts. Moreover, we also observe that the scale of the swarm (comparing 40 vs. 160 drones) does not affect the correctness of the state transitions, as the system consistently converges to the expected states in response to the events (with a slightly faster convergence in the larger swarm, since the space is more densely covered and the target is found more quickly)—see Figure 5.

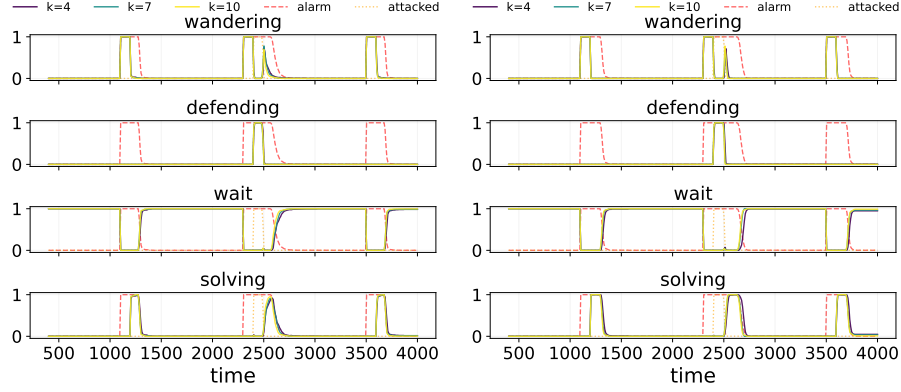


Fig. 5: The effect of execution frequency variability (smaller values denote higher variability; $k = 10$ is similar to a Dirac delta) on the evolution of the state distribution over time, for the unbounded history case and with network connectivity of 100m. Each line represents the average over N_{runs} independent runs. Left plot: 40 drones; right plot: 160 drones.

Resilience. The system exhibits strong resilience to variations in communication range and execution frequencies. Figure 5 compares the swarm behaviour under different Weibull shape parameters $k \in \{4, 7, 10\}$, which control the variability of the agents' execution rounds frequency ($k = 4$ implies high variability, $k = 10$ implies near-synchronous periodic execution). The nearly overlapping curves show that the convergence speed and stability of the collective states are largely unaffected by the asynchrony of the agents. This also demonstrates strong resilience under high relative mobility: even when large temporal gaps and spatial displacements occur between consecutive execution rounds (as in the high-variability $k = 4$ case), the swarm achieves consensus rapidly, with only a slightly longer transient phase compared to the more synchronous cases. Moreover, as shown in Figure 4, the system maintains low disagreement rates even with reduced communication range (comparing $R = 100$ m vs. $R = 75$ m), demonstrating robustness against network constraints. However, it is worth noting that a reduced communication radius increases the time required for the whole swarm to converge to a new state (as visible in the wider transient phases and higher peaks in the right plots of Figure 4). This is expected, as data takes longer to propagate through the network when the connectivity degree is lower.

Practical History Independence. Finally, the results show that the cFSM approach is history-independent in practice. The system correctly handles the state transitions without being hindered by the accumulation of past events, as evidenced by the stable bounded history size (bottom panels of Figure 4). It suggests that the compressed history mechanism effectively summarises nec-

essary context, allowing the system to work continuously without performance degradation or memory overflow, which is key for long-running missions.

5 Conclusion and Future Work

This paper proposes cFSMs, an extension to FSMs where states and transitions are *collective*, promoting macro-programming [3]. The abstract meta-model is operationalised in the aggregate computing model, and implemented as a library in the ScaFi aggregate programming language. Through a swarm robotics case study, it is shown that the approach enables the swarm to resiliently converge on the collective behaviour strategy, also under bounded memory assumptions.

This research opens up multiple directions of future work. A first extension to the practical expressiveness of cFSMs lies in supporting *parametric* states, able to encode and carry data. We also intend to show that the cFSM framework can be used to support multiple *design patterns* according to how the transition priorities are defined. Finally, the concurrency dimension of cFSMs can be explored by integrating with the notion of *aggregate processes* [2].

Acknowledgments. This work is partially supported by the Italian Science Fund (FIS3) Starting Grant project *FoMaSE – Foundations for Macro-programming-based Software Engineering* (Grant No. FIS-2024-00174, CUP J53C25002170001): <https://fis3-fomase.github.io/>.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aguzzi, G., Casadei, R., Viroli, M.: Macroswarm: A field-based compositional framework for swarm programming. *Log. Methods Comput. Sci.* **21**(3) (2025). [https://doi.org/10.46298/LMCS-21\(3:13\)2025](https://doi.org/10.46298/LMCS-21(3:13)2025)
2. Audrito, G., Casadei, R., Torta, G.: A general framework and decentralised algorithms for collective computational processes. *Future Gener. Comput. Syst.* **158**, 11–27 (2024). <https://doi.org/10.1016/j.future.2024.04.020>
3. Casadei, R.: Macro-programming multi-agent systems: A framework for artificial collective intelligence (blue sky ideas track). *International Foundation for Autonomous Agents and Multiagent Systems / ACM* (2026). <https://doi.org/10.65109/SAXG2906>
4. Casadei, R., Viroli, M.: Declarative macro-programming of collective systems with aggregate computing: An experience report. In: *Proceedings of the 26th International Symposium on Principles and Practice of Declarative Programming, PPDP 2024*. pp. 5:1–5:5. *ACM* (2024). <https://doi.org/10.1145/3678232.3678235>
5. Casadei, R., Viroli, M., Aguzzi, G., Pianini, D.: ScaFi: A Scala DSL and toolkit for aggregate programming. *SoftwareX* **20**, 101248 (2022). <https://doi.org/10.1016/J.SOFTX.2022.101248>

6. Corradini, F., Pettinari, S., Re, B., Rossi, L., Tiezzi, F.: A BPMN-driven framework for multi-robot system development. *Robotics and Autonomous Systems* **160**, 104322 (2023). <https://doi.org/10.1016/j.robot.2022.104322>
7. Cruz-Filipe, L., Montesi, F., Peressotti, M.: A formal theory of choreographic programming. *J. Autom. Reason.* **67**(2), 21 (2023). <https://doi.org/10.1007/S10817-023-09665-3>
8. Etheredge, M., Fahringer, T., Erlacher, F., et al.: Collaborative state machines: A better programming model for the cloud-edge-IoT continuum (2025). <https://doi.org/10.48550/ARXIV.2507.21685>
9. Hamann, H.: *Swarm Robotics: A Formal Approach*. Springer (2018)
10. Harel, D., Naamad, A.: The STATEMATE semantics of statecharts. *ACM Transactions on Software Engineering and Methodology* **5**(4), 293–333 (1996). <https://doi.org/10.1145/235321.235322>
11. Kohler, T.A., Bird, D., Wolpert, D.H.: Social scale and collective computation: Does information processing limit rate of growth in scale? *Journal of Social Computing* **3**(1), 1–17 (2022). <https://doi.org/10.23919/jsc.2021.0020>
12. Liu, W., Winfield, A.F.T.: Modeling and optimization of adaptive foraging in swarm robotic systems. *Int. J. Robotics Res.* **29**(14), 1743–1760 (2010). <https://doi.org/10.1177/0278364910375139>
13. Mytkowicz, T., Musuvathi, M., Schulte, W.: Data-parallel finite-state machines. In: *Architectural Support for Programming Languages and Operating Systems, ASPLOS’14*. pp. 529–542. ACM (2014). <https://doi.org/10.1145/2541940.2541988>
14. O’Keeffe, K.P., Hong, H., Strogatz, S.H.: Oscillators that sync and swarm. *Nature Communications* **8**(1) (2017). <https://doi.org/10.1038/s41467-017-01190-3>
15. Papadopoulou, M., Hildenbrandt, H., Hemelrijk, C.K.: A data-inspired framework to simulate collective behaviour of predator-prey systems. *Adaptive Behavior* **33**(5–6), 291–305 (2025). <https://doi.org/10.1177/10597123251364750>
16. Pianini, D., Montagna, S., Viroli, M.: Chemical-oriented simulation of computational systems with ALCHEMIST. *J. Simulation* **7**(3), 202–215 (2013). <https://doi.org/10.1057/jos.2012.27>
17. Spanoudakis, N.I.: Engineering multi-agent systems with statecharts: Theory and practice. *SN Comput. Sci.* **2**(4) (2021). <https://doi.org/10.1007/s42979-021-00706-5>
18. St-Onge, D., Varadharajan, V.S., Švogor, I., Beltrame, G.: From design to deployment: Decentralized coordination of heterogeneous robotic teams. *Frontiers in Robotics and AI* **7** (2020). <https://doi.org/10.3389/frobt.2020.00051>
19. Vidal, E., Thollard, F., de la Higuera, C., Casacuberta, F., Carrasco, R.: Probabilistic finite-state machines-part I. *IEEE Trans. Pattern Anal. Mach. Intell.* **27**(7), 1013–1025 (2005). <https://doi.org/10.1109/tpami.2005.147>
20. Viroli, M., Beal, J., Damiani, F., Audrito, G., Casadei, R., Pianini, D.: From distributed coordination to field calculus and aggregate computing. *J. Log. Algebraic Methods Program.* **109** (2019). <https://doi.org/10.1016/j.jlamp.2019.100486>
21. Wirsing, M., Jähnichen, S., Nicola, R.D.: Rigorous engineering of collective adaptive systems - 2nd special section. *Int. J. Softw. Tools Technol. Transf.* **25**(5), 617–624 (2023). <https://doi.org/10.1007/S10009-023-00734-X>
22. Zheng, L., Yang, J., Cai, H., Zhou, M., Zhang, W., Wang, J., Yu, Y.: MAgent: A many-agent reinforcement learning platform for artificial collective intelligence. *Proceedings of the AAAI Conference on Artificial Intelligence* **32**(1) (2018). <https://doi.org/10.1609/aaai.v32i1.11371>